

```
# Atelier noté – Conteneurisation Docker
## Séance 3 – Bloc DevOps, ESIEA
```

```
### 8 étapes
```

```
---
```

```
## Application fournie
```

Le dossier `starter-app/` de ce répertoire contient l'application Flask finale de la séance 2 (celle qui passe la CI), sans aucun fichier Docker :

```
...
```

```
starter-app/
├── app.py           # alert_threshold, sanitize_input, /health,
                    /status
├── test_app.py      # 4 tests déjà écrits (séance 2)
├── requirements.txt # flask, pytest, pytest-cov, flake8
└── .flake8
...
```

Copiez ce dossier dans votre dépôt de séance 1/2 – c'est le point de départ. Aucun fichier `Dockerfile` n'est fourni : c'est vous qui l'écrivez, à partir de l'étape 1.

```
## Prérequis
```

Docker Desktop (ou Docker Engine + Compose plugin) installé et fonctionnel sur votre machine – `docker version` et `docker compose version` doivent répondre sans erreur avant de commencer. Un accès Internet réel pour télécharger les images de base (`python:3.12`, `python:3.12-slim`, `redis:7-alpine`) : c'est indispensable, contrairement aux étapes purement Git/GitHub des séances précédentes, cet atelier a besoin d'un vrai daemon Docker qui télécharge de vraies images.

```
```bash
docker version
docker compose version
docker run --rm hello-world
```
```

La dernière commande doit afficher le message de bienvenue Docker – c'est la vérification de base que l'environnement est prêt avant de commencer.

```
---
```

```
## Étape 1 – Découverte & premier Dockerfile naïf
```

```
### Consignes
```

Intégrer `starter-app/` dans votre dépôt, puis écrire un premier `Dockerfile` volontairement simple : une seule étape, une image de base complète, sans se soucier pour l'instant de la taille ni de la sécurité –

ce sera l'objet des étapes suivantes. Ce premier jet doit rester le réflexe le plus direct : copier le code, installer les dépendances, démarrer l'application. Construire l'image avec `docker build`, lancer un conteneur avec `docker run` en pensant au port publié, puis vérifier que `/health` répond bien depuis l'extérieur du conteneur – un conteneur qui tourne sans erreur mais reste injoignable depuis l'hôte est l'erreur la plus fréquente à ce stade.

Étape 2 – Mesurer, utilisateur non-root, `.dockerignore`

Consignes

Mesurer réellement la taille de l'image avec `docker images`, puis l'observer couche par couche avec `docker history` pour repérer ce qui pèse le plus lourd. Corriger ensuite deux défauts hérités de l'étape précédente : le conteneur tourne en `root`, ce qui n'a rien d'anodin même en développement, et rien n'empêche `COPY . .` de copier des fichiers inutiles voire sensibles dans l'image. Créer un utilisateur applicatif dédié activé via l'instruction `USER` – attention à l'ordre des instructions, piège classique ici – et ajouter un `.dockerignore` limitant le contexte de build au strict nécessaire.

Étape 3 – Multi-stage build

Consignes

Réécrire le Dockerfile en deux stages : un premier stage, qu'on nomme couramment `builder`, qui installe les dépendances dans un environnement isolé, et un stage final, basé sur une image slim, qui ne récupère que ce qui est strictement nécessaire à l'exécution – sans compilateurs ni outils de build ni cache de paquets. C'est l'instruction `COPY --from=` qui permet de ne rapatrier que le nécessaire depuis le premier stage vers le second, sans jamais faire hériter l'image finale de son contenu superflu. Remplacer aussi le serveur de développement Flask par un vrai serveur WSGI, `gunicorn` : le serveur intégré de Flask (`debug=True`) ne doit jamais tourner dans une image destinée à autre chose que le poste du développeur – c'est aussi un signal de sécurité et de stabilité en production. Attention : chaque stage a son propre environnement, ce qui est déclaré dans le premier (comme une variable de chemin) ne survit pas automatiquement au second – un piège fréquent à ce stade.

Étape 4 – Mesurer le gain avant / après

Consignes

Comparer chiffres à l'appui la taille de l'image naïve (étapes 1-2) et celle du multi-stage (étape 3), avec `docker images` pour les deux, puis documenter le résultat dans votre README. C'est une mesure réelle sur

vosre machine, pas un chiffre à recopier d'ailleurs – les valeurs exactes dépendent des versions d'image du jour. Reconstituez les deux images juste avant de mesurer : comparer une ancienne image à une version modifiée du Dockerfile fausserait la comparaison. Un écart faible ou nul est en général le signe qu'un des deux Dockerfile n'est pas correctement écrit.

Étape 5 – Docker Compose multi-services

Consignes

L'application a maintenant besoin d'un second service : un compteur de visites (`/visits`) persistant, stocké dans Redis plutôt qu'en mémoire du conteneur `web` (sans quoi la valeur repartirait de zéro à chaque redémarrage). Écrire un `docker-compose.yml` qui orchestre le conteneur `web` (votre image) et un conteneur `redis` (image officielle), avec un réseau dédié pour que les deux services se voient par leur nom, et un volume nommé pour que les données Redis survivent à un arrêt des conteneurs. Réfléchissez à la façon dont `web` doit désigner Redis dans sa configuration réseau : ni `localhost`, ni une adresse IP fixe. Une fois le fichier écrit, `docker compose config` permet d'en vérifier la validité avant même de tenter un démarrage.

L'endpoint `/visits` doit être ajouté par vous-même dans `app.py` en vous inspirant de la structure des endpoints existants (`/health`, `/status`) – il doit incrémenter un compteur côté Redis et renvoyer sa valeur courante à chaque appel. Utiliser pour cela la fonction `get_redis_client()` : qui vous est fournie et qui permet de créer un client redis

Étape 6 – Healthcheck & dépendances entre services

Consignes

`depends\_on` seul ne garantit que l'ordre de *démarrage* des conteneurs, pas que Redis soit réellement prêt à accepter des connexions. Ajouter un `HEALTHCHECK` à l'image `web` – une vérification simple, sans dépendance à un outil qui pourrait être absent d'une image slim – et un healthcheck équivalent au service `redis`, puis remplacer le `depends\_on` simple par une version conditionnée sur `service\_healthy`. Le résultat attendu se lit dans `docker compose ps` : le statut des deux services doit passer à l'état healthy après quelques secondes, pas immédiatement au démarrage.

Étape 7 – Pousser l'image sur un registry

Consignes

Publier l'image `web` sur un registry de conteneurs (Docker Hub ou `ghcr.io`), avec un tag versionné en plus de `latest` – se limiter à `latest` interdit ensuite de revenir précisément à une version antérieure en cas de problème. Une authentification (`docker login`, ou son équivalent pour `ghcr.io`) est nécessaire avant de pouvoir pousser quoi que ce soit. C'est ce qui rendra l'image récupérable depuis n'importe quelle machine – y compris le serveur de déploiement de la séance 4 – et il vaut mieux vérifier vous-même, en supprimant l'image locale puis en la retéléchargeant, qu'elle est bien récupérable depuis zéro.

Étape 8 – Consolidation & README

Consignes

Documenter dans le README l'ensemble de ce qui a été construit aujourd'hui : comment builder l'image (`docker build`), comment lancer la stack complète en une seule commande (`docker compose up`), et où trouver l'image publiée sur le registry – lien ou nom exact, pas un exemple générique laissé tel quel. Quelqu'un qui découvre votre dépôt pour la première fois doit pouvoir tout relancer sans avoir à vous poser de question. Relire l'ensemble des changements de la séance et vérifier la checklist de livrable ci-dessous avant de considérer le travail terminé.

Checklist de livrable (à vérifier avant de partir)

`Dockerfile` multi-stage présent, produisant une image sensiblement plus légère que la version naïve initiale (mesure chiffrée documentée dans le README, pas juste affirmée). Conteneur exécuté en utilisateur non-root (vérifié avec `whoami`, pas seulement la présence de la ligne `USER`). `.dockerignore` limitant le contexte de build aux fichiers réellement nécessaires. `HEALTHCHECK` fonctionnel sur l'image `web`, visible comme `healthy` dans `docker compose ps`. `docker-compose.yml` orchestrant au moins deux services (`web` + `redis`) avec un réseau dédié, un volume nommé pour la persistance, et `depends\_on` avec `condition: service\_healthy`. Endpoint `/visits` fonctionnel, dont la valeur survit à un redémarrage du seul conteneur `web`. Image publiée sur un registry, taguée avec un numéro de version explicite en plus de `latest`, et récupérable depuis zéro (`docker pull` après `docker rmi` local vérifié). README à jour avec les instructions de build et de lancement.

En cas de blocage

`docker build` qui échoue sur `FROM` avec une erreur réseau : vérifier la connexion Internet de la machine et que Docker Desktop est bien démarré – contrairement aux séances précédentes, cette séance a besoin d'un accès réel aux registries d'images. Un conteneur qui démarre puis s'arrête aussitôt (`docker compose ps` affiche `Exited`) : consulter `docker compose logs <service>` en premier réflexe, l'erreur exacte y est presque toujours explicite. Un `HEALTHCHECK` qui reste bloqué sur `starting` indéfiniment : vérifier que la commande de test fonctionne bien *depuis

l'intérieur* du conteneur avant de blâmer la configuration Compose. Une image poussée sur le registry mais introuvable ensuite : vérifier la visibilité du package sur le registry choisi, parfois privée par défaut. Un `/visits` qui repart toujours de 1 après un `docker compose restart web` : signe d'un problème de connexion au bon service Redis, ou d'un volume de persistance mal monté – vérifier directement le contenu stocké côté Redis.